

Documentatie - Proiect ISP

APLICATIE PENTRU GESTIONAREA VOLUNTARILOR LA ONG

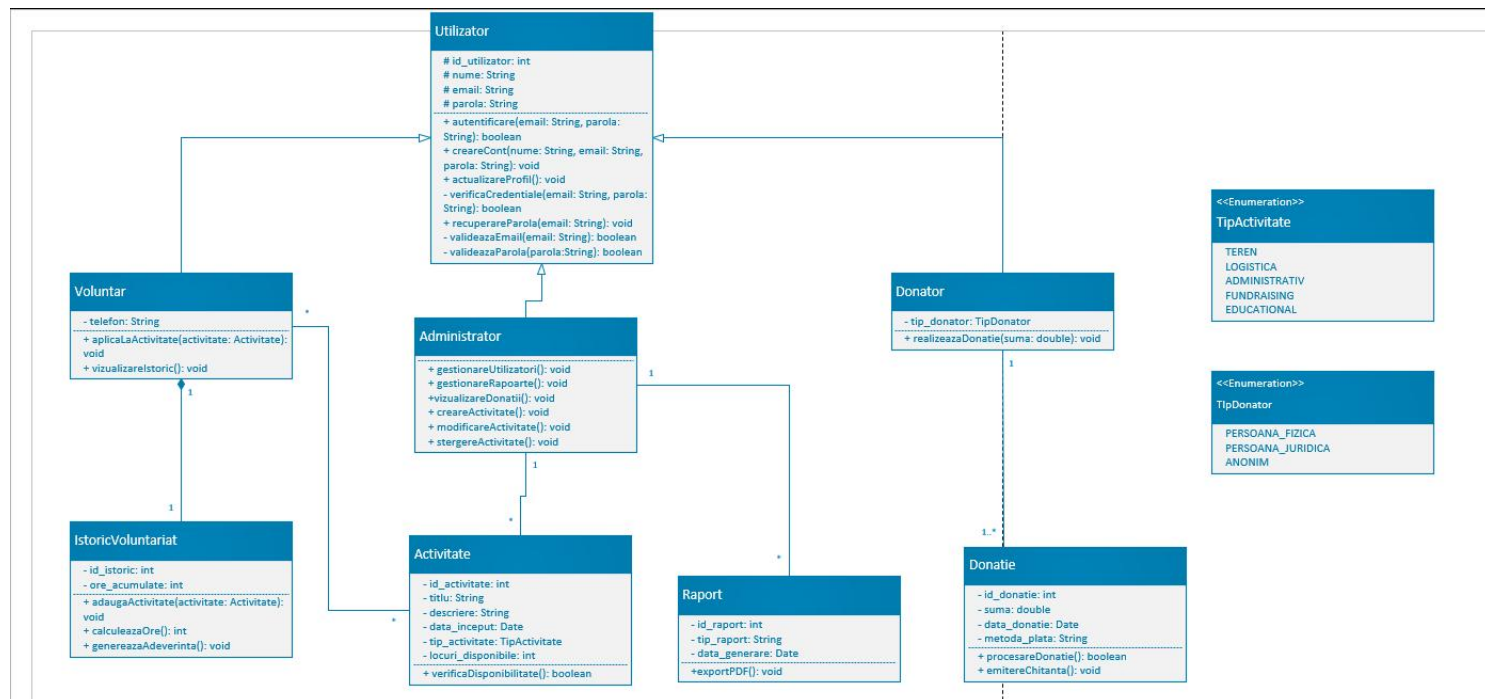
Nume: Șova Ioan-Rareș
Grupa: 332AA

Descriere:

Proiectul reprezintă un sistem software dezvoltat în Java, destinat administrării activităților unui ONG. Aplicația permite gestionarea a trei tipuri principale de utilizatori: Administratori (care creează activități și rapoarte), Voluntari (care se pot înscrie la activități și pot genera adeverințe cu istoricul de voluntariat) și Donatori (care pot realiza donații pentru a susține ONG-ul). Pentru a asigura calitatea codului și respectarea arhitecturii, proiectul este organizat în două pachete distincte (models și tests). Funcționalitățile critice sunt testate automatizat folosind framework-ul JUnit. De asemenea, pentru exportul rapoartelor și adeverințelor a fost integrată biblioteca externă iTextPDF.

1. Partea I - Lucru in echipa

1.1. Diagrama de clase



Descrierea modelului:

Diagrama de clase modelează structura sistemului și conține cele trei tipuri de relații obligatorii studiate:

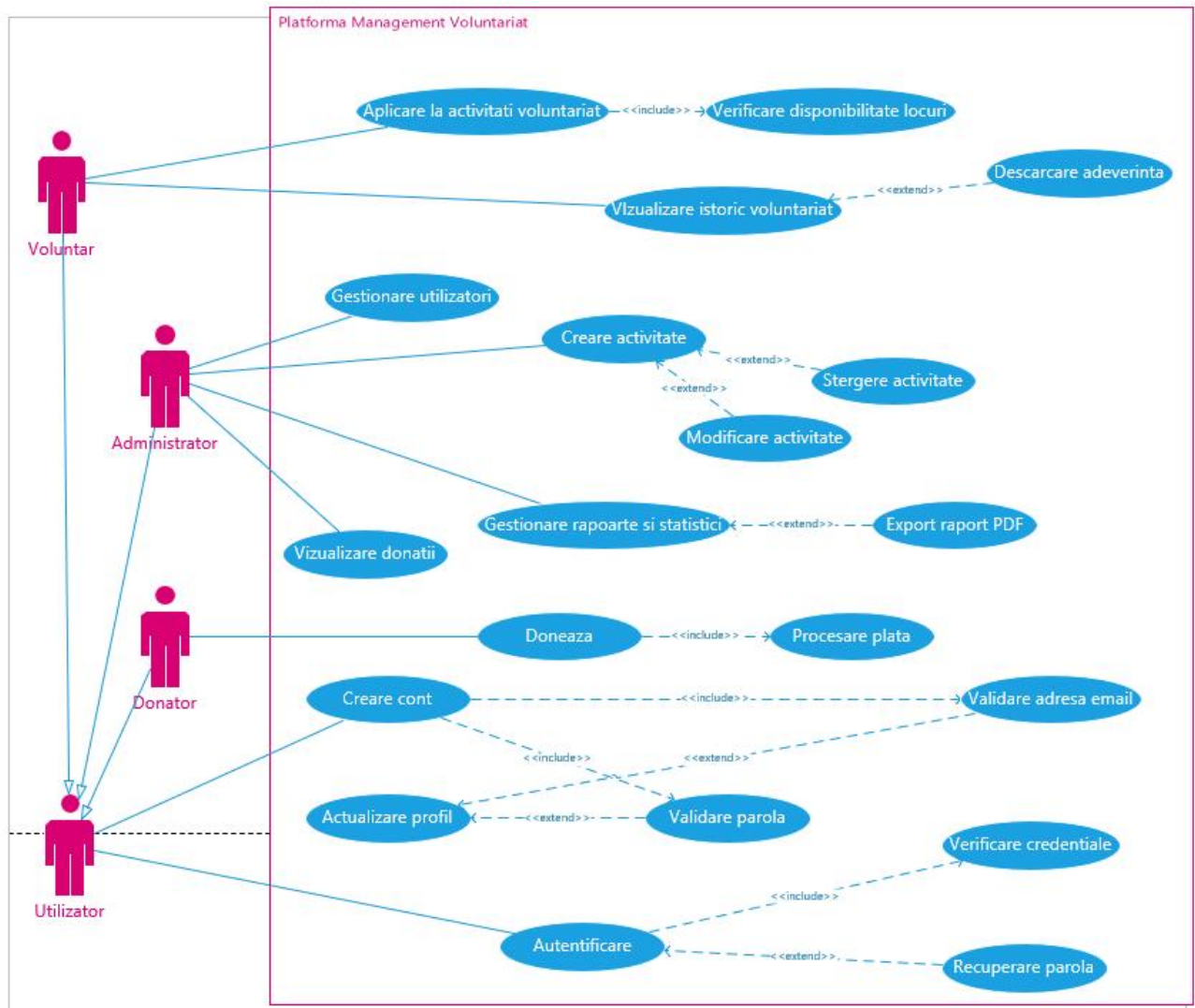
-Generalizare (Moștenire): Clasa abstractă Utilizator este extinsă de clasele derivate Administrator, Voluntar și Donator. Toate moștenesc attributele de bază (id_utilizator, nume, email, parola) și comportamentele comune (autentificare, creare cont).

-Compoziție: Există o relație strânsă între Voluntar și IstoricVoluntariat (1 la 1), istoricul fiind instanțiat direct în constructorul voluntarului, așadar ciclul de viață al istoricului depinde de cel al voluntarului.

-Asociere: Clasa Administrator este asociată cu gestionarea claselor Activitate și Raport. Relații de asociere există între Donator și Donatie (un donator are o listă de donații asociate, 1 la 1..*), dar și între Voluntar și Activitate (un voluntar poate participa la mai multe activități, iar o activitate are o listă de voluntari).

-Enumerări: Sunt folosite enums precum TipActivitate (TEREN, LOGISTICA, etc.) și TipDonator.

1.2 Diagrama cazurilor de utilizare



Descrierea modelului:

Diagrama surprinde interacțiunea actorilor cu sistemul:

-*Actori*: Utilizator (actor general), Voluntar, Administrator, Donator.

-*Cazuri de utilizare* principale: Autentificare, Creare cont (comune tuturor).

Administratorul are acces la: Creare/Modificare/Stergere activitate, Gestionare utilizatori și Gestionare rapoarte. Voluntarul poate: Aplica la activități, Vizualizare și descărcare adeverinta. Donatorul poate realiza donații.

-*Relații <<include>> și <<extend>>*:

* Include: "Donează" include "Procesare plată".

"Aplicare la activități voluntariat" include obligatoriu "Verificare disponibilitate locuri".

*Extend: "Creare activitate" poate fi extinsă opțional de "Modificare activitate" sau "Ștergere activitate".

"Vizualizare istoric voluntariat" poate fi extins de "Descărcare adeverință" .

2. Partea II - Lucru individual

2.1. Diagrama de activități

Descrierea modelului:

Această diagramă detaliază cazul de utilizare „Aplicare la activități voluntariat” și modelează funcționalitatea metodei aplicaLaActivitate(Activitate) din clasa Voluntar.

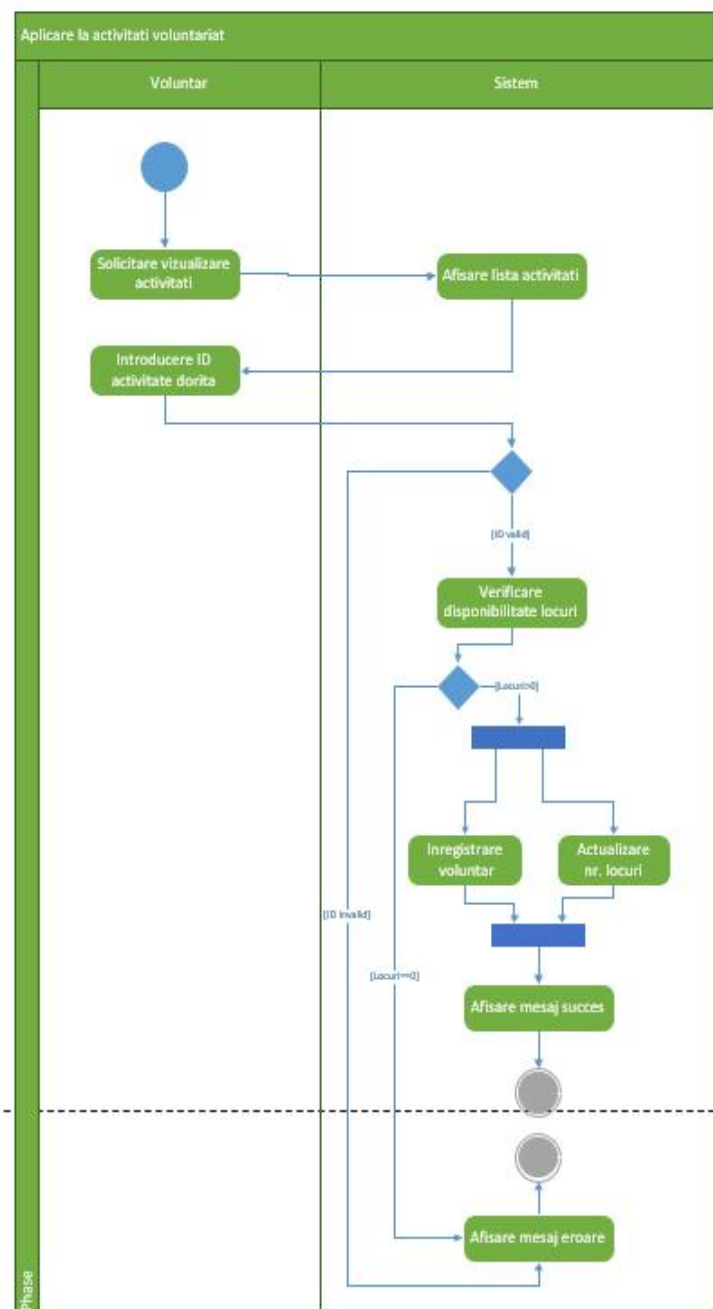
-Fluxul principal: Voluntarul solicită afișarea activităților, iar sistemul afișează lista.

Utilizatorul introduce ID-ul dorit. Sistemul validează ID-ul, apoi verifică locurile disponibile

-Decizii (Noduri de decizie):

* Dacă se găsesc locuri disponibile ([Locuri > 0]), firul de execuție se desparte (Fork) pentru a executa în paralel: adăugarea voluntarului în listă și actualizarea numărului de locuri. Acestea se reunesc (Join) într-un afișaj de succes.

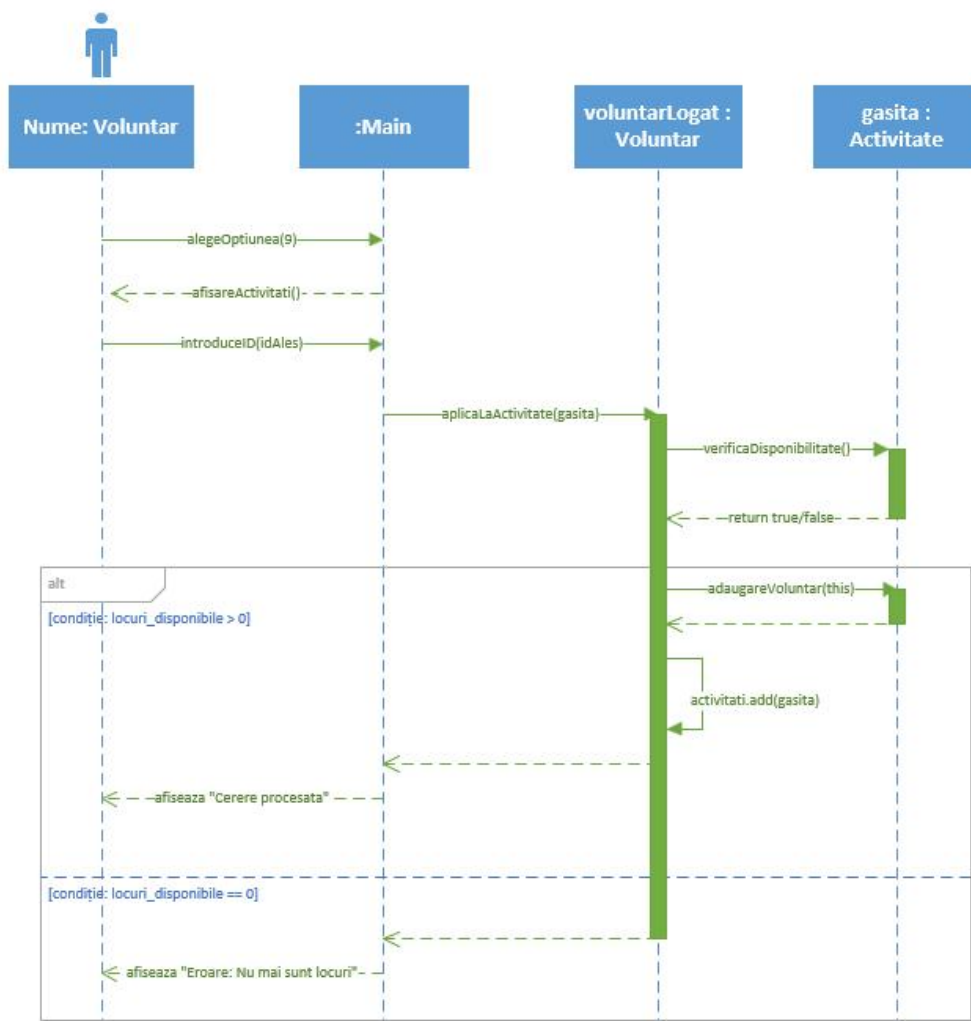
* Dacă nu sunt locuri ([Locuri == 0]), sistemul ramifică direct către afișarea unui mesaj de eroare ("Nu mai sunt locuri disponibile"), finalizând fluxul alternativ.



Precizare: Acest flux complex a fost testat unitar în clasa `AplicareActivitateTest.java` (folosind JUnit). Pentru validarea acestui flux, clasa `AplicareActivitateTest.java` implementează 5 seturi de date de test (situații posibile), acoperind:

1. Situația normală (succes, înscriere cu locuri disponibile suficiente).
2. Limita inferioară (înscriere când mai există 1 singur loc disponibil).
3. Forțarea excepției (înscriere respinsă la 0 locuri disponibile).
4. Situație anormală de sistem (verificare comportament când locurile sunt pe negativ).
5. Verificarea ramurii decizionale pentru utilizator deja înscris (sistemul blochează duplicarea și nu scade locurile)."

2.2 Alta diagramă UML (Diagrama de Secvență)



Descrierea modelului:

Această diagramă reprezintă implementarea aceluiași caz de utilizare, „Aplicare la activități voluntariat”, detaliind interacțiunea dinamică la nivel de obiecte și apeluri de metode.

-Obiecte participante: Actorul (Voluntar), meniul principal (:Main), instanța logată (voluntarLogat : Voluntar) și activitatea căutată (gasita : Activitate).

-Schimbul de mesaje: Utilizatorul alege opțiunea 9 în Main, introduce ID-ul, iar controlul este pasat către metoda aplicaLaActivitate(gasita) a clasei Voluntar.

-Blocul logic alt: Diagrama reprezintă vizual instrucțiunea condițională din cod.

*Dacă apelul verificaDisponibilitate() returnează true (locuri > 0), se apelează metoda adaugareVoluntar(this) pe obiectul activitate, iar activitatea este adăugată în lista voluntarului.

*Dacă returnează false (locuri = 0), se afișează direct eroarea, respingând aplicarea.

3. Referințe bibliografice

1. Curs și Laborator "Ingineria Sistemelor de Programe", UNSTPB, Facultatea de Automatica si Calculatoare, 2026.
2. Documentația oficială Java (Oracle) - Java Platform, Standard Edition Documentation:
<https://docs.oracle.com/en/java/>
3. Documentația bibliotecii iText pentru generarea rapoartelor PDF în Java:
<https://itextpdf.com/resources>
4. Documentația oficială JUnit 5 (JUnit Jupiter) pentru testarea automatizată a aplicațiilor Java:
<https://junit.org/junit5/docs/current/user-guide/>